ПРИКЛАДНОЙ УРОВЕНЬ

Прикладной уровень

- Теоретические и практические аспекты сетевых приложений
  - модели обслуживания транспортного уровня
  - клиент-серверная архитектура
  - одноранговая архитектура
- Изучение протоколов прикладного уровня на примере самых популярных из них:
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
- Разработка сетевых приложений
  - API сокетов

Примеры сетевых приложений

- электронная почта
- всемирная паутина (WWW)
- текстовые сообщения
- удаленный доступ
- одноранговый файлообмен
- многопользовательские онлайн-игры
- потоковое видео (YouTube, Netflix)
- VOIP-технология (Skype)
- видеоконференции в режиме реального времени
- социальные сети
- поисковые системы
- …

Создание сетевого приложения

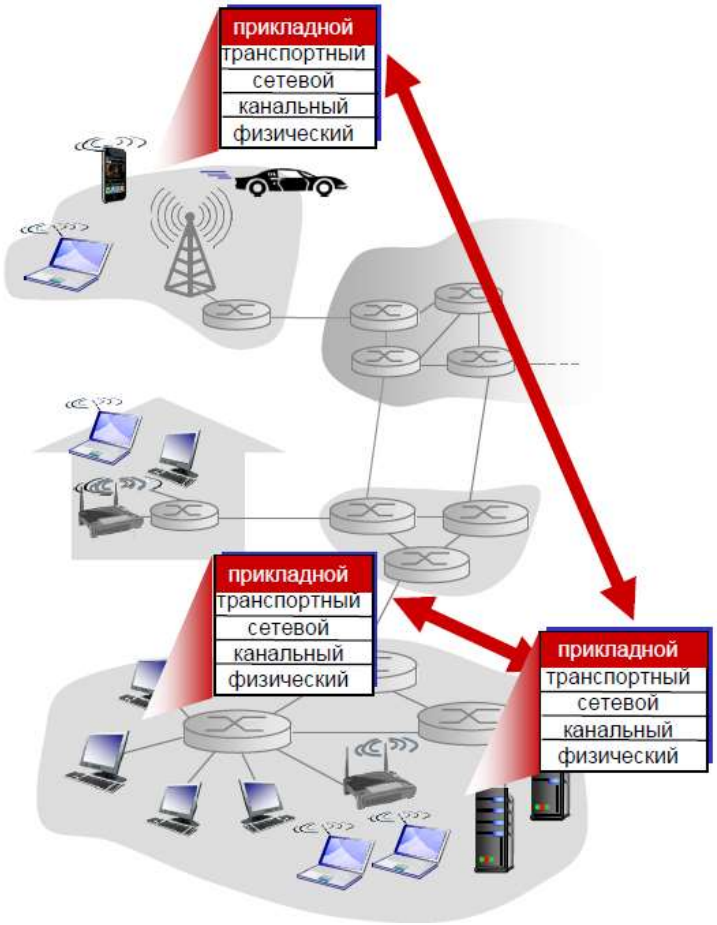
Взаимодействие сетевых приложений происходит между конечными системами на прикладном уровне

Разработка программ, которые:

- запускаются на разных конечных системах
- взаимодействуют по сети
  - пример взаимодействия – ПО веб-сервера и ПО браузера

Нет необходимости в разработке ПО для устройств ядра сети

- устройства ядра сети не используют приложения
  - как следствие, разработка и развертывание приложений на конечных системах происходит еще быстрее



Архитектура сетевого приложения

Архитектура приложений и архитектура сети — это разные вещи

Доступные архитектуры приложений:

- **Клиент-серверная**. Существует один хост, называемый **сервером**, который постоянно находится в режиме онлайн и обслуживает запросы от других многочисленных хостов, называемых **клиентами**
  - Всемирная паутина (WWW), FTP, электронная почта
  - Центры обработки данных (дата-центры) – образуют мощный виртуальный сервер

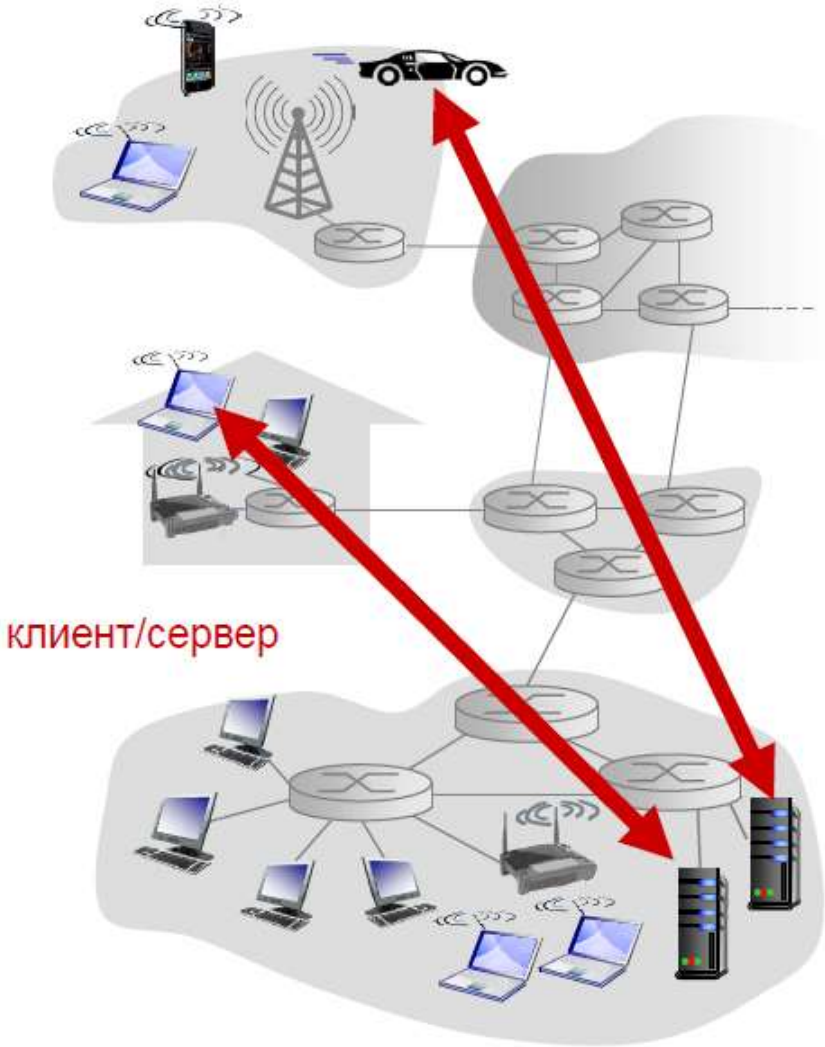
- **Одноранговая (P2P)**. Приложения используют непосредственное взаимодействие между парой соединенных хостов, называемых пирами
  - BitTorrent, IP-телефония, IP-телевидение

- Гибридные архитектуры, где совмещаются клиент-серверный и одноранговый подходы.
  - Например, во многих мессенджерах серверы используются для отслеживания IP-адресов пользователей, но сообщения от пользователя к пользователю посылаются напрямую между двумя хостами (не проходя через промежуточные серверы)

Клиент-серверная архитектура

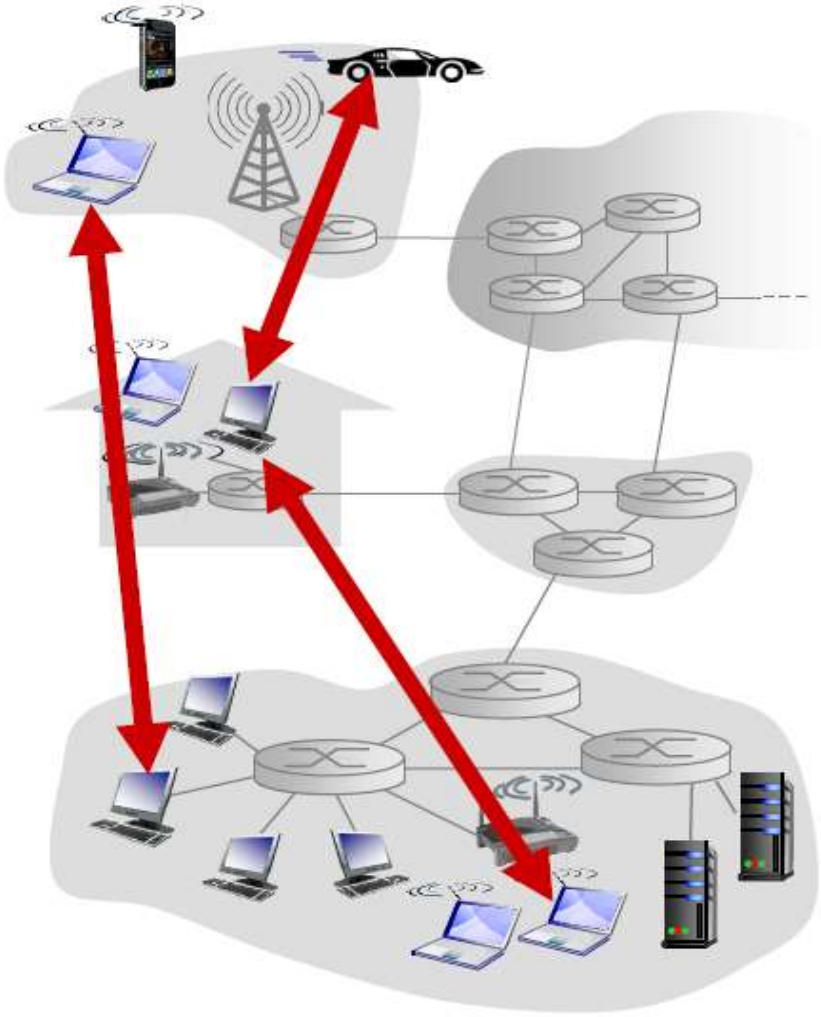
- Сервер:
  - хост всегда в режиме онлайн
  - есть постоянный IP-адрес
  - центры обработки данных с возможностью масштабирования (когда одной машины не достаточно)

- Клиенты:
  - взаимодействуют с сервером
  - периодические соединения
  - могут иметь динамические IP-адреса
  - не взаимодействуют друг с другом напрямую



Одноранговая (P2P) архитектура

- Нет постоянного сервера в режиме онлайн
- Произвольные конечные системы взаимодействуют напрямую
- Пиры запрашивают услуги у других, предоставляя сами услуги другим пирам
  - **само-масштабируемость** – новые пиры увеличивают как объем, так и потребность в обслуживании
- Взаимодействие периодическое и возможна смена адресов
  - сложнее управляемость

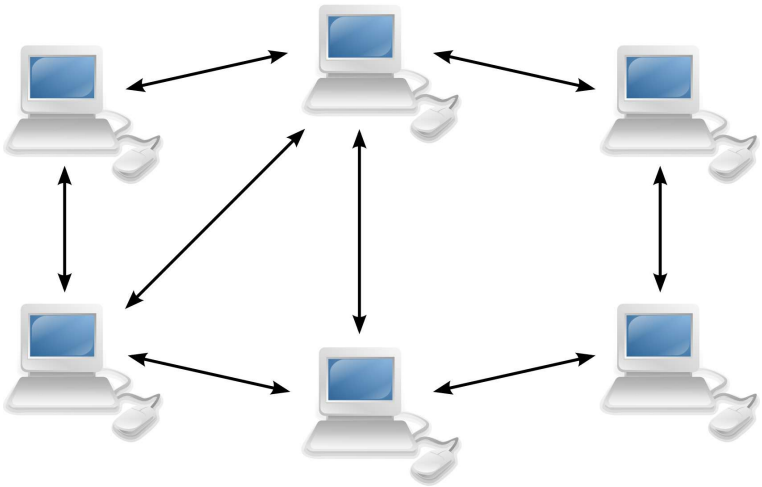


Проблемы в P2P приложениях

- Адаптация к сетям доступа
  - Большинство резидентных сетей доступа построены по асимметричному принципу, когда **скорость входящего трафика значительно превышает скорость исходящего**

- Безопасность
  - Сети в связи с их широким распространением и открытой природой представляют собой проблему безопасности

- Стимулирование
  - Успешное будущее одноранговых сетей связано с задачей привлечения пользователей, которые добровольно будут предлагать свои ресурсы вычисления, хранения и передачи данных для приложений



Взаимодействие процессов

Процесс: программа, запущенная на хосте

- Межпроцессное взаимодействие между двумя процессами одного хоста (определяется операционной системой)
- Процессы разных хостов взаимодействуют, обмениваясь сообщениями
- В приложениях, использующих одноранговую архитектуру, процесс может выступать как в качестве клиентского, так и серверного

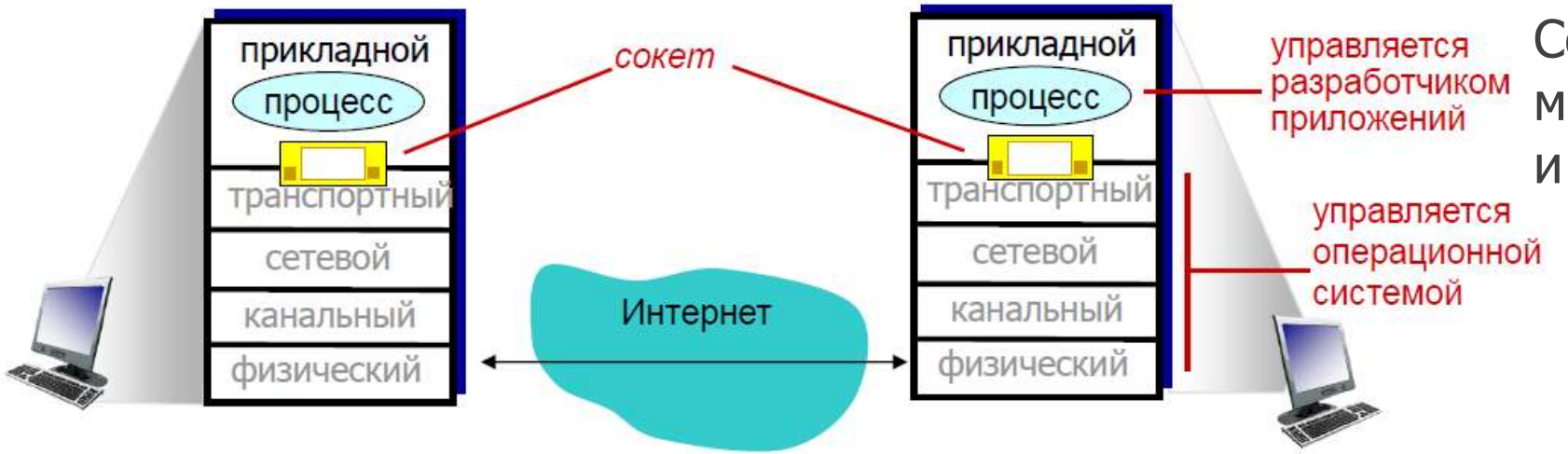
клиенты, серверы

клиентский процесс:
инициирующий взаимодействие

серверный процесс:
ожидающий взаимодействия

Сокеты

- Процесс отправляет/получает сообщения через свой сокет (программный интерфейс)
- Сокет – аналог «двери»
  - передающий процесс «проталкивает» сообщение в «дверь»
  - передающий процесс полагается на транспортную инфраструктуру по ту сторону «двери», доставляющую сообщения в сокет принимающего процесса
- Разработчик приложения управляет всем, что находится со стороны прикладного уровня сокета, но со стороны транспортного уровня он способен только контролировать:
  - выбор транспортного протокола
  - некоторые параметры транспортного уровня, такие как максимальный буфер и максимальный размер сегмента



Сокет – это API между приложением и сетью

Факультет математики и компьютерных наук

Адресация процессов

- Для получения сообщений процесс должен иметь идентификатор
- Уникальный 32-битный адрес – у каждого хоста
- Вопрос: достаточно ли IP-адреса хоста для идентификации процесса?
- Ответ: **нет**, на одном хосте может быть запущено много процессов
- Идентификатор включает как IP-адрес, так и номера портов, связанные с процессом
- Примеры номеров портов:
  - HTTP-сервер: 80
  - почтовый сервер: 25

Транспортные службы, требуемые приложению

- Целостность данных
  - некоторые приложения (передача файлов, веб-операции) требуют 100% надежности в передаче данных
  - другие приложения (аудио) допускают некоторые потери
- Время доставки
  - для эффективной работы некоторых приложений (Интернет-телефония, онлайн-игры) требуется низкий уровень задержек
- Пропускная способность
  - определенный минимальный порог требуется для работы некоторых приложений (мультимедиа)
  - другие приложения (*эластичные*) работают, используя любой канал, не предъявляя особых требований к полосе пропускания
- Безопасность
  - шифрование, целостность данных

Требования к транспортным службам, предъявляемые сетевыми приложениями

приложение	потери данных	пропускная способность	чувствительность к времени доставки
передача файлов	не допускаются	эластичное	нет
эл. почта	не допускаются	эластичное	нет
веб-приложения	не допускаются	эластичное	нет
аудио/видео реального времени	допустимы	аудио: 5 Кбит/с – 1 Мбит/с видео:10 Кбит/с – 5 Мбит/с	да, сотни миллисекунд
сохраненное аудио/видео	допустимы	как предыдущее	да, несколько секунд
интерактивные онлайн-игры	допустимы	от нескольких Кбит/с	да, сотни миллисекунд
текстовые сообщения	не допускаются	эластичное	да и нет

Транспортные службы Интернета

Службы TCP:

- с установлением соединения (рукопожатие): логическое соединение между клиентским и серверным процессами
- надежная доставка от передающего процесса к принимающему
- контроль перегрузки: принудительное снижение скорости передачи при перегрузке сети
- контроль потока: отправитель не «завалит» получателя сообщениями
- не обеспечивается: безопасность, гарантированное время доставки и пропускная способность

Службы UDP:

- ненадежная передача данных между процессами
- не обеспечивается: надежность, контроль потока, контроль перегрузки, гарантированное время доставки, пропускная способность, безопасность, установление соединения

- **Транспортные протоколы сегодняшнего Интернета не могут обеспечить гарантированное время доставки и о пропускную способность**

Безопасность протокола TCP

- TCP и UDP
  - без шифрования
  - текстовые пароли передаются в сокет открытым текстом
- SSL (Secure Sockets Layer)
  - зашифрованное TCP-соединение
  - целостность данных
  - конечная аутентификация
- SSL реализован на прикладном уровне
  - приложения используют SSL для общения с TCP
- SSL имеет свой API сокета
  - передающий процесс отправляет обычные текстовые данные в сокет SSL
  - SSL на передающем хосте шифрует данные и передает их в сокет TCP
  - зашифрованные данные отправляются по сети в сокет TCP к принимающему процессу
  - сокет получателя передает зашифрованные данные в SSL, затем расшифровывает их
  - SSL передает обычные текстовые данные через свой сокет в принимающий процесс

Интернет-приложения и используемые протоколы

приложение	протокол прикладного уровня	базовый транспортный протокол
электронная почта	SMTP [RFC 2821]	TCP
удаленный доступ	Telnet [RFC 854]	TCP
Всемирная паутина	HTTP [RFC 2616]	TCP
передача файлов	FTP [RFC 959]	TCP
потоковые мультимедиа	HTTP (например, YouTube),	TCP или UDP
Интернет-телефония	RTP [RFC 1889] SIP, RTP, частный (например, Skype)	TCP или UDP

Протокол прикладного уровня

Протокол прикладного уровня определяет:

- Типы передаваемых сообщений:
  - запрос, ответ
- Синтаксис сообщения:
  - содержание и формат полей в сообщении
- Семантика сообщения:
  - значение информации в полях
- Правила: определяют, когда и как процессы передают и получают сообщения

Открытые протоколы:

- описываются документами RFC
- примеры: HTTP, SMTP

Проприетарные протоколы:

- пример: Skype

Всемирная паутина и HTTP

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Всемирная паутина и HTTP

Всемирная паутина стала главным Интернет приложением с начала 90-х гг. Принцип ее функционирования — **работа по запросу**: пользователи получают то, что они хотят и когда им это требуется, в отличие от традиционных радио- и телевещания

Терминология WWW:

- Веб-страница состоит из объектов
- Объектом может служить файл HTML, изображение JPEG, аудиофайл,…
- Веб-страница включает основной HTML-файл, в котором присутствует несколько объектов, на которые он ссылается
- У каждого объекта есть свой URL-адрес

www.someschool.edu/someDept/pic.gif

Имя хоста

Путь к файлу

Обзор HTTP

HTTP: протокол передачи гипертекста

- протокол прикладного уровня для веб-приложений
- работает поверх TCP

Клиент-серверная модель

- клиент: браузер запрашивает, получает и отображает веб-объекты (используя протокол HTTP)
- сервер: веб-сервер отправляет объекты (используя протокол HTTP) в ответ на запросы

Обзор HTTP

- Использует TCP:
  - клиент инициирует TCP-соединение (создает сокет) с сервером на порту 80
  - сервер принимает TCP-соединение от клиента
  - браузер (HTTP-клиент) и веб-сервер (HTTP-сервер) обмениваются сообщениями прикладного уровня
  - TCP-соединение закрывается
- HTTP – протокол «без сохранения состояния» (stateless protocol)
  - сервер не хранит информацию о последних запросах

HTTP-соединения

Как отправлять каждую пару запрос-ответ: через отдельное соединение либо через одно и то же TCP-соединение?

- Непостоянные соединения. Каждое TCP- соединение передает ровно одно сообщение-запрос и одно сообщение-ответ
  - один объект - через одно соединение
  - соединение затем закрывается
  - загрузка нескольких объектов требует нескольких соединений
- Постоянные соединения
  - через одно соединение клиент-сервер можно отправить много объектов

Непостоянное HTTP-соединение

www.someSchool.edu/someDepartment/home.index *(содержит текст, ссылки на 10 jpeg-изображений)*

1а. HTTP-клиент инициирует TCP-соединение с HTTP-сервером (с его процессом) хоста www.someSchool.edu на порту 80

1б. HTTP-сервер хоста www.someSchool.edu, ожидающий TCP-соединения на порту 80, “принимает” соединение, уведомляя клиента

2. HTTP-клиент отправляет *сообщение-запрос* (содержащее URL-адрес) в сокет соединения TCP. Сообщение указывает, что клиент запрашивает объект someDepartment/home.index

3. HTTP-сервер получает сообщение-запрос, формирует *ответное сообщение,* содержащее запрашиваемый объект, и отправляет сообщение в свой сокет

время

Непостоянное HTTP-соединение

4. HTTP-сервер закрывает TCP-соединение

5. HTTP-клиент получает ответное сообщение, содержащее html-файл, и отображает его. Обрабатывая файл, находит ссылки на 10 jpeg-объектов

время

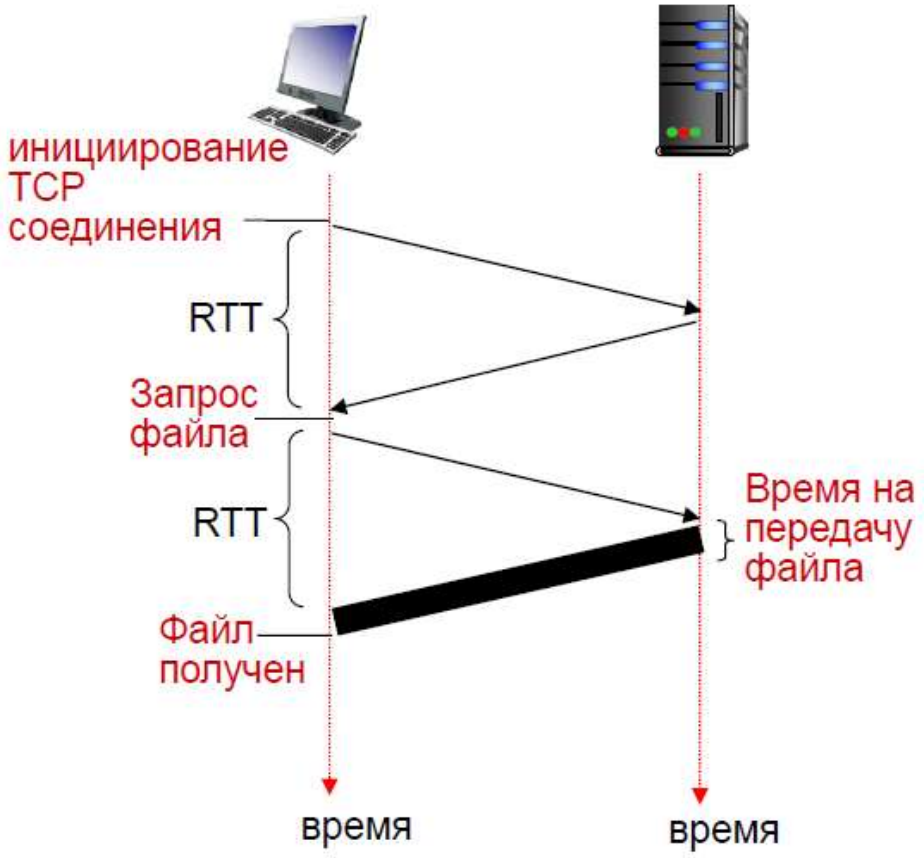
6. Шаги 1-5 повторяются для каждого из 10 jpeg-объектов

Непостоянное HTTP-соединение: время ответа

RTT (round-trip time) - время прохождения пути от клиента до сервера и обратно для небольшого пакета

Время ответа HTTP:

- один RTT на инициирование TCP-соединения
- один RTT на HTTP-запрос и первые несколько байт HTTP-ответа
- время на передачу файла
- время ответа для непостоянного HTTP-соединения=2RTT+ время на передачу файла



Постоянное HTTP-соединение

- Непостоянное HTTP-соединение:
  - требуется 2 RTT на 1 объект
  - нагрузка на ОС для каждого нового соединения
  - браузеры часто открывают параллельные TCP-соединения для извлечения объектов по ссылкам
- Постоянное HTTP-соединение:
  - после отправки ответа сервер оставляет соединение открытым
  - через открытое соединение можно отправить последовательность HTTP-сообщений между клиентом и сервером
  - клиент отправляет запрос как только встретит содержащий ссылку объект
  - всего 1 RTT для каждого из объектов

Сообщение-запрос HTTP

- Два вида HTTP-сообщений: запрос, ответ

- Сообщение-запрос HTTP:
  - ASCII (воспринимаемый человеком формат)

Символ возврата каретки

Символ перевода строки

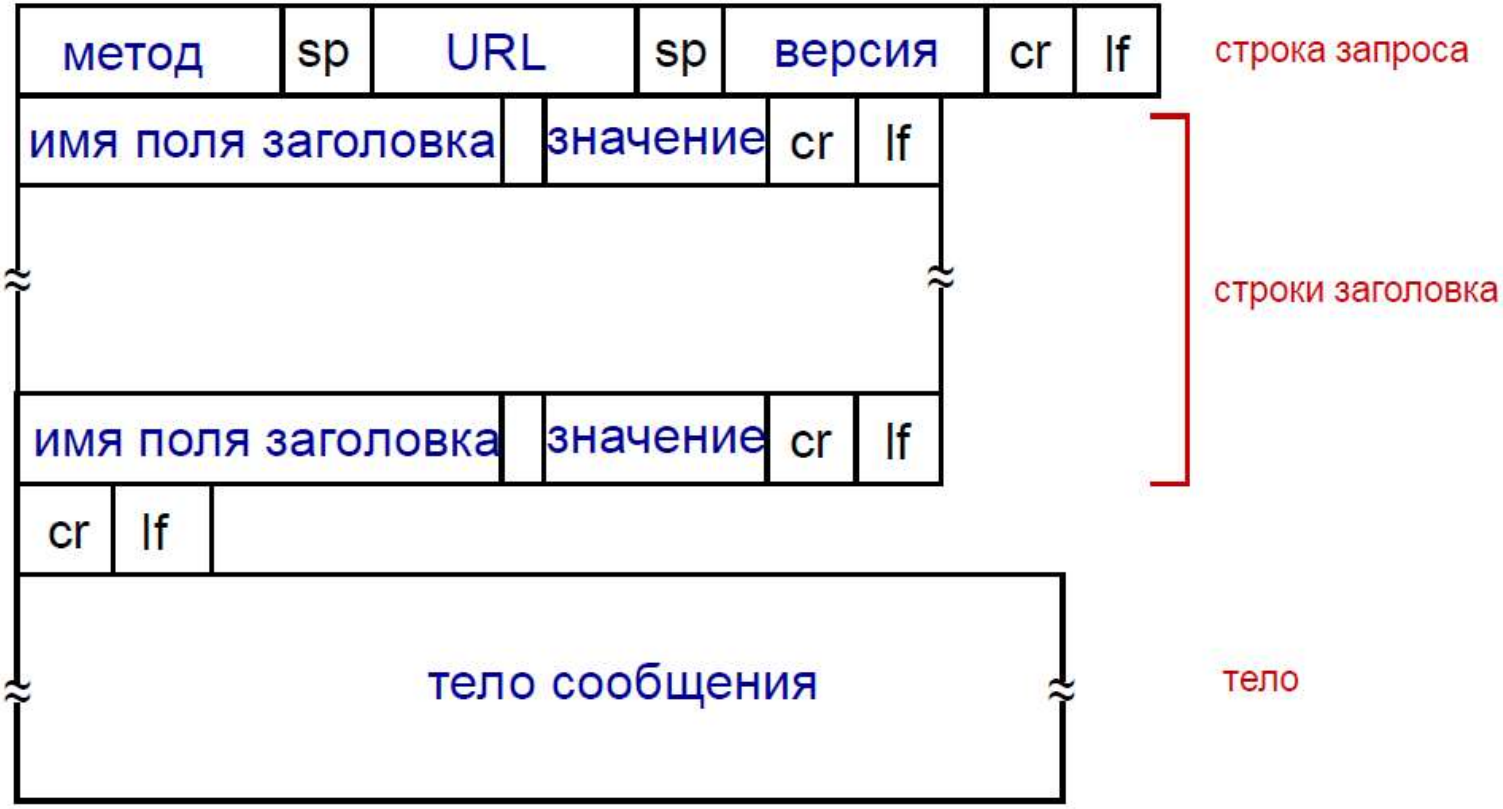
Строка запроса (команды GET, POST, HEAD)

Строки заголовка

Возврат каретки и перевод строки в начале строки указывают на конец заголовка

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

Основной формат HTTP-запроса



Ввод данных через формы

- Метод POST:
  - веб-страница часто содержит формы ввода
  - введенные данные загружаются на сервер в тело сообщения

- URL:
  - использует метод GET
  - введенные данные загружаются в поле URL строки запроса

Основные методы

- HTTP/1.0:
  - GET
  - POST
  - HEAD
    - запрос, при котором тело сообщения в ответе остается пустым (полезно при отладке)
- HTTP/1.1:
  - GET, POST, HEAD
  - PUT
    - работает как update для указанного объекта
  - DELETE
    - удаляет объект, указанный в поле URL

Ответное сообщение HTTP

строка состояния
(код состояния протокола и
сообщение состояния)

строки заголовка

данные, например
запрошенный
HTML-файл

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
   GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
   1\r\n
\r\n
данные данные данные данные...
```

Коды состояния HTTP-ответа

Коды появляются в первой строке ответа от сервера

- Некоторые примеры:

200 OK

- запрос выполнен успешно , запрошенный объект следует дальше в этом же сообщении

301 Moved Permanently

- запрошенный объект окончательно перемещен, его новое место расположения указано далее в сообщении (Location:)

400 Bad Request

- сервер не понял запрос (скорее всего, синтаксическая ошибка)

404 Not Found

- запрошенный документ на сервере не найден

505 HTTP Version Not Supported

- сервер не поддерживает указанную в запросе версию протокола

Состояние сеанса пользователь-сервер: cookie

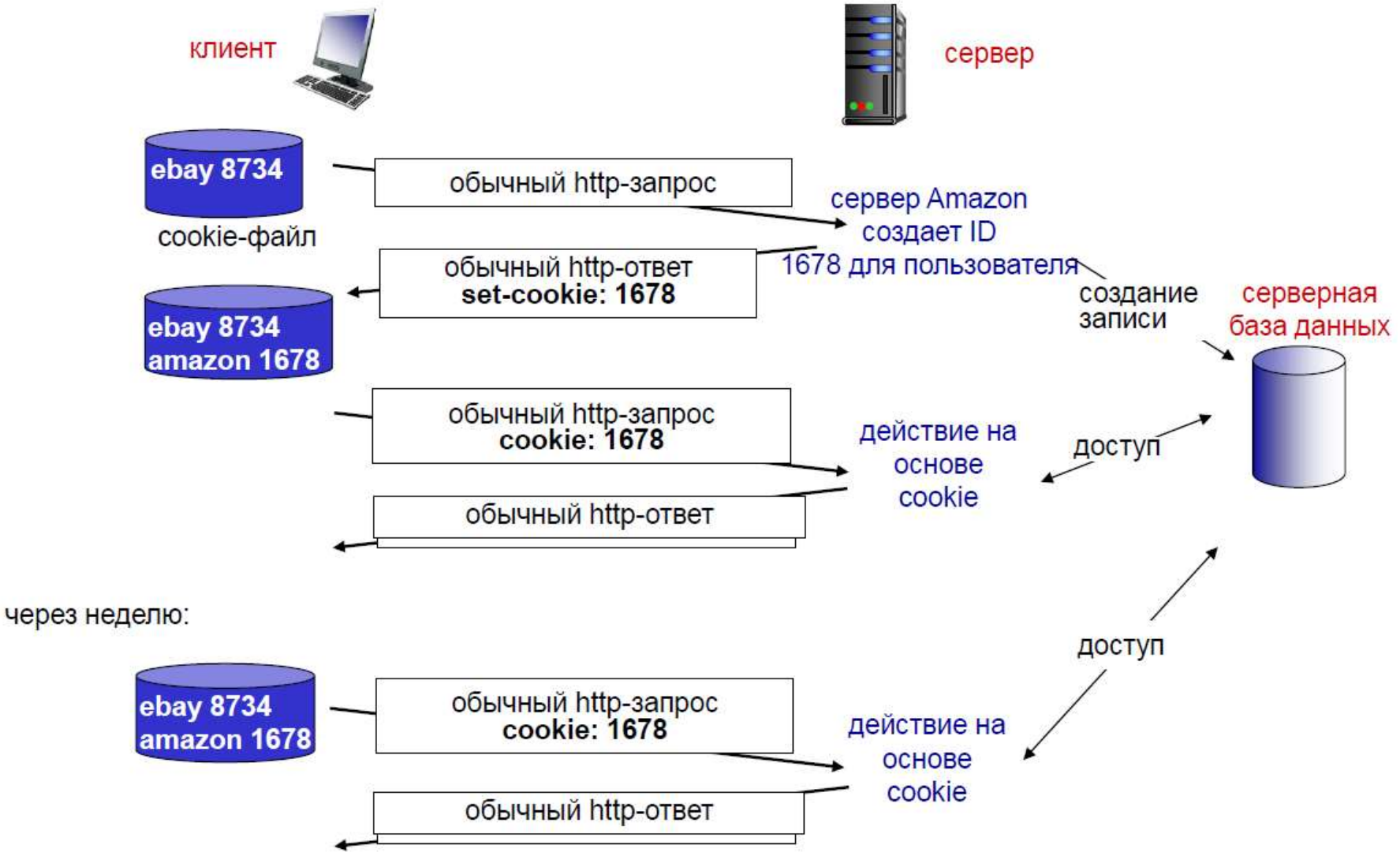
Многие веб-сайты используют механизм cookie. Куки позволяет веб-сайтам отслеживать состояние пользовательского соединения

Четыре компонента cookie:

- строка заголовка в HTTP-ответе
- строка заголовка в HTTP-запросе
- cookie-файл на хосте пользователя, управляемый его браузером
- база данных на стороне веб-сервера

Можно сказать, что механизм cookie создает как бы дополнительный сеансовый уровень поверх протокола HTTP, который не сохраняет информацию о состоянии соединения

Пример:

- Маша выходит в Интернет со своего домашнего ПК
- первый раз заходит на сайт электронных покупок
- когда начальный HTTP-запрос поступает на сайт, веб-сайт создает:
  - уникальный ID
  - запись в базе данных сервера, соответствующую этому ID

Состояние сеанса пользователь-сервер: cookie